Lezione 9 - Same Origin Policy e CORS

Introduzione : dal metodo OPTIONS alla sicurezza del browser

Nella lezione precedente abbiamo visto come, utilizzando strumenti come **cURL**, sia possibile inviare direttamente richieste HTTP a un server e osservare in modo esplicito metodi, header e risposte.

Abbiamo anche introdotto il metodo **OPTIONS**, **chiarendo che spesso non è l'utente a inviarlo, ma è il browser a farlo automaticamente come parte dei meccanismi di sicurezza del Web moderno.**

Per comprendere **perché il browser si comporta in questo modo**, è necessario introdurre due concetti fondamentali:

- la **Same-Origin Policy**;
- il meccanismo **CORS (Cross-Origin Resource Sharing)**.
  Questi concetti spiegano **perché alcune richieste funzionano con cURL ma non dal browser**, e perché il metodo OPTIONS è così strettamente legato alla sicurezza delle applicazioni web.

La Same-Origin Policy

La Same-Origin Policy (SOP) è una politica di sicurezza implementata dai browser che limita le interazioni tra risorse provenienti da origini diverse.

In termini semplici:

> **una pagina web può effettuare richieste solo verso la stessa origine da cui è stata caricata, salvo esplicite autorizzazioni.**

Quando due URL hanno la stessa origine

Due URL sono considerati della **stessa origine** solo se **tutti e tre** questi elementi coincidono:

1. **protocollo (`http` / `https`);**
2. **dominio;**
3. **porta.**

Esempi

✅ Stessa origine

Arduino

```
https://example.com/page1
https://example.com/page2
```

Origini diverse

- `https://example.com` → `http://example.com` *(protocollo diverso)*
- `https://example.com` → `https://api.example.com` *(sottodominio diverso)*
- `https://example.com` → `https://example.com:8080` *(porta diversa)*

Perché esiste la Same-Origin Policy

La Same-Origin Policy è uno dei **pilastri della sicurezza del Web**.
Il suo obiettivo **è proteggere l'utente da comportamenti malevoli.**

Senza questa politica, un sito potrebbe:

- **leggere dati sensibili di altri siti aperti nel browser;**
- **inviare richieste non autorizzate a nome dell'utente (ad esempio usando cookie o token);**
- **accedere a risorse private o riservate.**

In altre parole, la SOP **impedisce che una pagina web possa "spiare" o manipolare risorse appartenenti ad altre origini.**

Cos'è CORS (Cross-Origin Resource Sharing)

CORS è un meccanismo che permette di **rilassare in modo controllato** la Same-Origin Policy.

Non è il browser a decidere liberamente:

- **è il server che dichiara quali origini sono autorizzate ad accedere alle sue risorse.**

Con CORS, il server può specificare:

- **da quali domini accetta richieste;**
- **quali metodi HTTP sono consentiti;**
- **quali header possono essere inviati;**
- **se le credenziali sono ammesse.**

Queste informazioni vengono comunicate tramite **header HTTP di risposta**, come:

- `Access-Control-Allow-Origin`
- `Access-Control-Allow-Methods`
- `Access-Control-Allow-Headers`

CORS: un'eccezione controllata alla Same-Origin Policy

Quindi **CORS (Cross-Origin Resource Sharing):**

- **è il meccanismo che consente a un server di dichiarare esplicitamente quali origini sono autorizzate ad accedere alle sue risorse.**

In altre parole:

> **la Same-Origin Policy non viene eliminata**, ma può essere **rilassata in modo controllato** su decisione del server.

Il browser si limita a **far rispettare** queste regole, senza mai decidere autonomamente.

Simple Requests (Richieste Semplici)

Non tutte le richieste cross-origin sono considerate rischiose.
Alcune vengono classificate come Simple Requests e vengono inviate direttamente, senza alcun controllo preliminare.

Esempio di Simple Request

JS JavaScript

```javascript
fetch('http://localhost:3001/users', { method: 'GET' });
```

Criteri per una Simple Request

Una richiesta è considerata *simple* se soddisfa **tutti** questi requisiti:

1. **Metodo HTTP**:
   - `GET`
   - `HEAD`
   - `POST`
2. **Headers consentiti**:
   - `Content-Type: text/plain`
   - `application/x-www-form-urlencoded`
   - `multipart/form-data`

Esempio:

JS JavaScript

```javascript
fetch('/api/login', {
  method: 'POST',
  body: 'username=marco&password=123',
  headers: { 'Content-Type': 'application/x-www-form-urlencoded' }
});
```

Questa **è una Simple Request**, anche se modifica lo stato del server.
Chiariamo subito questo punto:

> **non è la modifica dello stato del server in sé a far scattare il preflight.**

Se la richiesta rientra in questi criteri, il browser **non invia alcuna preflight request**.

ⓘ Perche `POST` non viene considerato un metodo pericoloso

La risposta sta nel fatto che, **per il browser**, la distinzione non è basata sulla semantica REST, ma su **criteri storici e di sicurezza pratica**.

1. **“Pericoloso” per REST ≠ “pericoloso” per il browser**
   Dal punto di vista REST:

- `GET` → sicuro (*safe*, read-only)
- `POST` → **modifica lo stato**
- `PUT` → modifica lo stato

- `PATCH` → modifica lo stato
- `DELETE` → modifica lo stato
  Quindi potremmo pensare che `POST` modifica lo stato quanto `PUT` e `DELETE`, tuttavia ciò che fa scattare il preflight request non è la modifica dello stato in sé.
  Difatti la ragione di questo comportamento lo si deve guardare non dal lato server ma da lato client, più specificatamente dal lato del browser il quale non ragiona in termini REST.

2. **Il criterio del browser è storico (HTML)**
   Il concetto di *Simple Request* nasce **prima delle API REST**, ed è legato a ciò che **HTML è sempre stato in grado di fare senza JavaScript**.
   Storicamente, una pagina HTML poteva:

HTML

```html
<form method="GET" action="/search">
<form method="POST" action="/login">
```

GET e POST sono sempre stati permessi
PUT, PATCH, DELETE non sono mai stati supportati dai form HTML

Questo significa che:

- milioni di siti hanno sempre inviato `POST` cross-origin
- bloccarli avrebbe **rotto il Web.**

Ovviamente anche `POST` è "sicuro" fin tanto che rispetta tutti i criteri delle Simple Requests

Esempio in cui scatta la preflight request con `Post`

JavaScript

```javascript
fetch('/api/users', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json' //Header che non rispetta i criteri delle Simple Requests
},
    body: JSON.stringify({ name: 'Mario' })
});
```

```
8
```

Richieste non semplici e Preflight Request

Quando una richiesta **esce dai criteri delle Simple Requests, il browser la considera potenzialmente pericolosa.**

In questi casi entra in gioco il meccanismo delle **Preflight Requests**.

Esempio di richiesta che genera preflight

JS JavaScript

```javascript
fetch('http://localhost:3001/users', {
  method: 'DELETE',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer token123'
  }
});
```

Motivi del preflight:

1. metodo non "semplice" ;
   - `PUT`
   - `PATCH`
   - `DELETE`
2. header personalizzati:
   - `Authorization`
   - `X-Custom-Header`
   - `Content-Type: application/json`
3. Oppure entrambi:
   - Esempio tipico:

JS JavaScript

```javascript
fetch('/api/users/1', {
  method: 'DELETE',
```

```
  headers: {
    'Authorization': 'Bearer token',
    'Content-Type': 'application/json'
  }
});
```

Qui **Preflight obbligatoria**, perché:

- il metodo è “pericoloso”;
- l’header `Authorization` può esporre dati sensibili.

Cos’è una Preflight Request

Una **Preflight Request**, quindi, è una richiesta HTTP **OPTIONS** che il browser invia **automaticamente prima della richiesta reale del client**.

Il suo scopo è semplice:

> **verificare preventivamente se il server autorizza una determinata operazione cross-origin.**

In altre parole, il browser sta chiedendo al server:

> ***“Posso davvero eseguire questa richiesta, con questo metodo e questi header, provenendo da questa origine?”***

L’utente o lo sviluppatore **non la scrive manualmente**:

- **è il browser a gestire tutto il processo.**
  Il browser, quindi, **non si fida ciecamente e chiede prima il permesso al server.**

Flusso concettuale (alto livello)

Immaginiamo questa chiamata nel codice JavaScript:

JS JavaScript

```
fetch('/api/users', { method: 'DELETE' });
```

Dal punto di vista del browser:

1. **Il client intende eseguire un’operazione potenzialmente pericolosa (`DELETE`);**

2. **Il browser decide di non inviare subito la richiesta**
3. **Invia invece una richiesta OPTIONS (preflight);**
4. **Attende la risposta del server;**
5. **Solo se il server autorizza l'operazione, invia la richiesta originale**.

Cosa contiene una Preflight Request

La richiesta **OPTIONS** inviata dal browser contiene informazioni fondamentali, tra cui:

- **l'origine della richiesta (`Origin`);**
- **il metodo HTTP che il client vorrebbe usare (`Access-Control-Request-Method`);**
- **gli header personalizzati che verranno inviati (`Access-Control-Request-Headers`).**
  Esempio concettuale:

```http
OPTIONS /api/users HTTP/1.1
Origin: https://myapp.com
Access-Control-Request-Method: DELETE
```

Cosa deve fare il server

Il server deve rispondere esplicitamente alla preflight request, indicando cosa è consentito.

Esempio di risposta corretta:

```http
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://myapp.com
Access-Control-Allow-Methods: GET, POST, DELETE
Access-Control-Allow-Headers: Authorization, Content-Type
```

Questa risposta equivale a dire:

> **"Sì, accetto richieste DELETE da questa origine, con questi header."**

Solo dopo il browser invia la richiesta reale

Se (e solo se) la risposta alla preflight è positiva, il browser procede con la richiesta originale:

HTTP

```http
DELETE /api/users HTTP/1.1
Origin: https://myapp.com
Authorization: Bearer abc123
```

Se invece il server non risponde correttamente o nega il permesso, la richiesta reale non viene mai inviata.

Flusso completo di una Preflight Request

Scenario:
Immaginiamo di avere una chiamata di questo tipo

JavaScript

```javascript
fetch('http://localhost:3001/users/1', {
  method: 'DELETE',
  headers: { 'Authorization': 'Bearer abc123' }
});
```

Cosa accade “dietro le quinte”

STEP 1 – Preflight automatico del browser

HTTP

```http
OPTIONS /users/1 HTTP/1.1
Host: localhost:3001
Origin: http://localhost:3000
Access-Control-Request-Method: DELETE
Access-Control-Request-Headers: Authorization
```

Il browser chiede:

- “Posso usare DELETE?”
- “Posso inviare l’header Authorization?”
- “Da questa origine?”

STEP 2 – Risposta del server

```http
HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://localhost:3000
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Authorization, Content-Type
```

Il server autorizza esplicitamente l'operazione.

STEP 3 – Invio della richiesta reale
Solo dopo una risposta positiva al preflight, il browser invia la richiesta effettiva:

```http
DELETE /users/1 HTTP/1.1
Host: localhost:3001
Origin: http://localhost:3000
Authorization: Bearer abc123
```

Header CORS principali

Durante questo scambio, entrano in gioco alcuni header fondamentali:

Header inviati dal browser (preflight)

- `Origin`:
  - **Indica l'origine della richiesta, cioè da dove proviene il codice client che sta effettuando la chiamata.**
  - L'**origin** è composta da:
    - **protocollo (`http` / `https`)**
    - **dominio**
    - **porta**

> ⚠ **Nota:**
>
> - `Origin` **non è impostato manualmente dal client JavaScript;**
> - **viene aggiunto automaticamente dal browser;**
> - **strumenti come `cURL` o `Postman` non applicano CORS e possono inviarlo solo manualmente.**

- `Access-Control-Request-Method`:

    - **Specifica quale metodo HTTP il client intende utilizzare nella richiesta reale.**
    - **Se il metodo non è incluso nell'elenco restituito dal server, il browser blocca la richiesta reale.**
- `Access-Control-Request-Headers`:
    - **Indica quali header HTTP personalizzati il client intende inviare nella richiesta reale.**
    - **Se l'header non è presente nell'elenco consentito dal server, il browser blocca l'operazione prima dell'invio.**

Header restituiti dal server

- `Access-Control-Allow-Origin`:
    - **Indica quali origini (origin) sono autorizzate ad accedere alla risorsa.**
    - Può essere:
        - un'origin specifica (`http://localhost:3000`)
        - `*` (tutte le origin, **solo se non si usano credenziali**)
- `Access-Control-Allow-Methods`:
    - **Specifica quali metodi HTTP il server consente per le richieste cross-origin(`GET` , `POST` , `PUT` , etc.)**
    - **Se il metodo non è elencato, la richiesta reale non verrà inviata.**
    - **Questo header è fondamentale nelle preflight request.**
- `Access-Control-Allow-Headers`:
    - **Indica quali header HTTP personalizzati il client è autorizzato a inviare.**
    - **Se l'header non è presente nell'elenco il browser blocca la richiesta.**
- `Access-Control-Allow-Credentials`:
    - **Indica se il server consente l'invio di credenziali nella richiesta.**
    - Per credenziali si intendono:
        - cookie;
        - header `Authorization`;
        - certificati client-side.

> ⓘ **Regole fondamentali**

- **se è `true`, il browser può inviare credenziali;**
- **se è `false` o assente, le credenziali vengono bloccate;**
- **non può essere usato insieme a `Access-Control-Allow-Origin: *`**

- `Access-Control-Max-Age`:
  - **Indica per quanto tempo (in secondi) il risultato della preflight request può essere memorizzato in cache dal browser.**
  - **È particolarmente utile in applicazioni con molte chiamate ripetute.**

Questi header determinano **se il browser è autorizzato a inviare la richiesta reale oppure se deve bloccarla per motivi di sicurezza**.

In sintesi:

Le **Simple Requests** (come la maggior parte delle `GET`):

- **vengono inviate direttamente dal browser perché utilizzano metodi e header considerati a basso rischio.**

Le richieste che usano metodi non sicuri (`PUT`, `PATCH`, `DELETE`) o header non standard fanno invece scattare una Preflight Request (OPTIONS).

In questi casi, il browser **chiede preventivamente al server se l'operazione è consentita**, prima di inviare la richiesta reale.